

Bérlők CRUD műveletei

Bérlők létrehozás:

A bérlő létrehozása funkció fejlesztése során elkészítettem az új bérlők adatainak rögzítésére szolgáló API végpontot. A rendszer képes eltárolni a bérlő személyes adatait, például nevét, email címét, telefonszámát és lakcímét. A fejlesztés során kialakítottam az adatbázisba történő adatmentés folyamatát és a szükséges SQL utasításokat. A végpont sikeres mentés esetén visszajelzést küld a kliens oldal számára. A működést Postman segítségével ellenőriztem és teszteltem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    userid,
    nev,
    email,
    telefonszam,
    személyiszam,
    lakcim
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `INSERT INTO berlok
      (userid, nev, email, telefonszam, személyiszam, lakcim)
      VALUES (?, ?, ?, ?, ?, ?)` ,
      [
        userid,
        nev,
        email,
        telefonszam,
        személyiszam,
        lakcim
      ]
    );

  }
});
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
        res.json({
            siker: true,
            uzenet: "Berlo sikeresen létrehozva!"
        });

    } catch (err) {

        res.status(500).json(err);

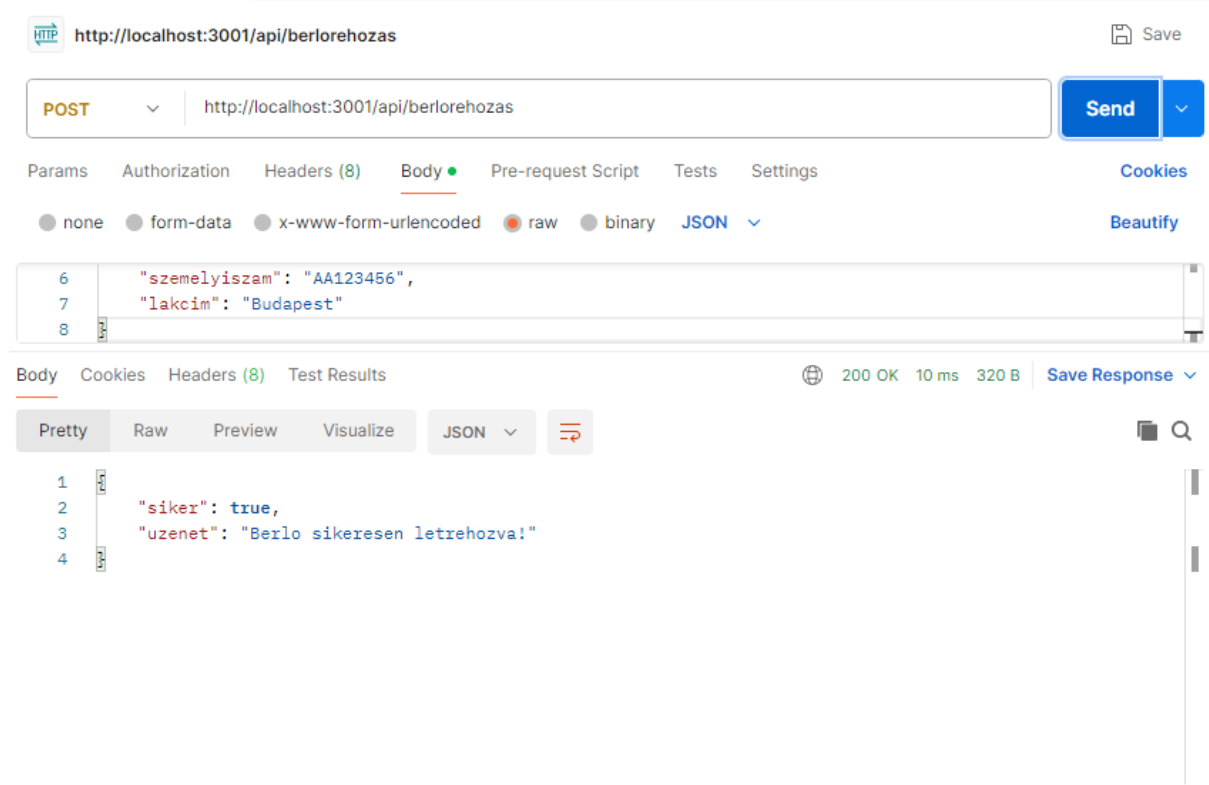
    } finally {

        if(conn) conn.release();

    }

});

module.exports = router;
```



1. Részlet az `berlolethozas.js` fájlból.

Bérlők módosítása:

A bérlő módosítása funkció elkészítése során lehetővé tettem a bérlők adatainak frissítését az adatbázisban. A rendszer képes a kiválasztott bérlő személyes adatainak módosítására és mentésére. A fejlesztés során létrehoztam az adatbázis frissítéséhez szükséges SQL

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

lekérdezéseket. A módosítás sikeres végrehajtása esetén a rendszer visszajelzést küld a felhasználó számára. A funkció helyes működését Postman tesztekkel ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

  const {
    nev,
    email,
    telefonszam,
    személyiszam,
    lakcim
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE berlok
      SET
      nev = ?,
      email = ?,
      telefonszam = ?,
      személyiszam = ?,
      lakcim = ?
      WHERE id = ?`,
      [
        nev,
        email,
        telefonszam,
        személyiszam,
        lakcim,
        req.params.id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Berlo modositva!"
    });

  } catch (err) {
```

```
        res.status(500).json(err);

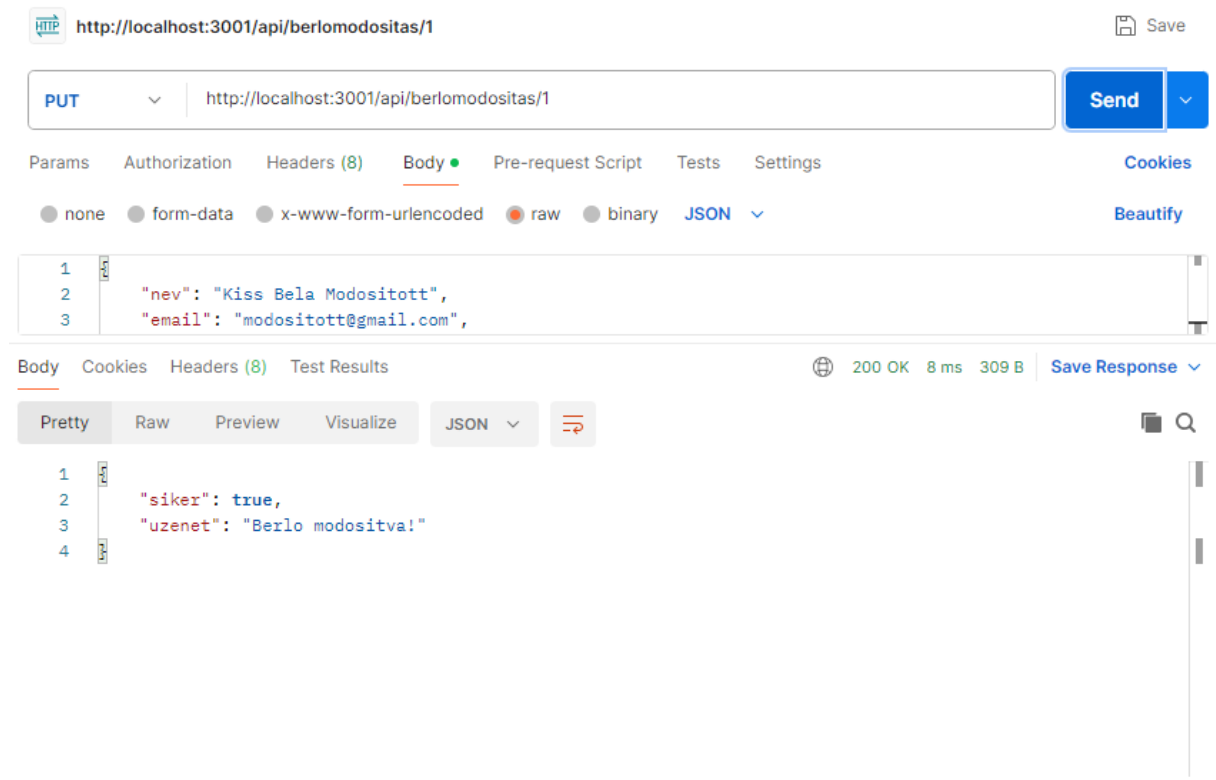
    } finally {

        if(conn) conn.release();

    }

});

module.exports = router;
```



1. Részlet az berlomodositas.js fájlból.

Bérlők törlése:

A bérlő törlése funkció fejlesztése során elkészítettem a kiválasztott bérlő rekordjának eltávolítására szolgáló API végpontot. A rendszer az azonosító alapján megkeresi és törli a megfelelő adatot az adatbázisból. A fejlesztés során figyelmet fordítottam a biztonságos adatkezelésre és a hibakezelésre is. Sikeres törlés esetén a végpont tájékoztató üzenetet küld vissza a kliensnek. A funkció működését Postman alkalmazás segítségével teszteltem és ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
router.delete("/:id", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      "DELETE FROM berlok WHERE id = ?",
      [req.params.id]
    );

    res.json({
      siker: true,
      uzenet: "Berlo torolve!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {

    if(conn) conn.release();

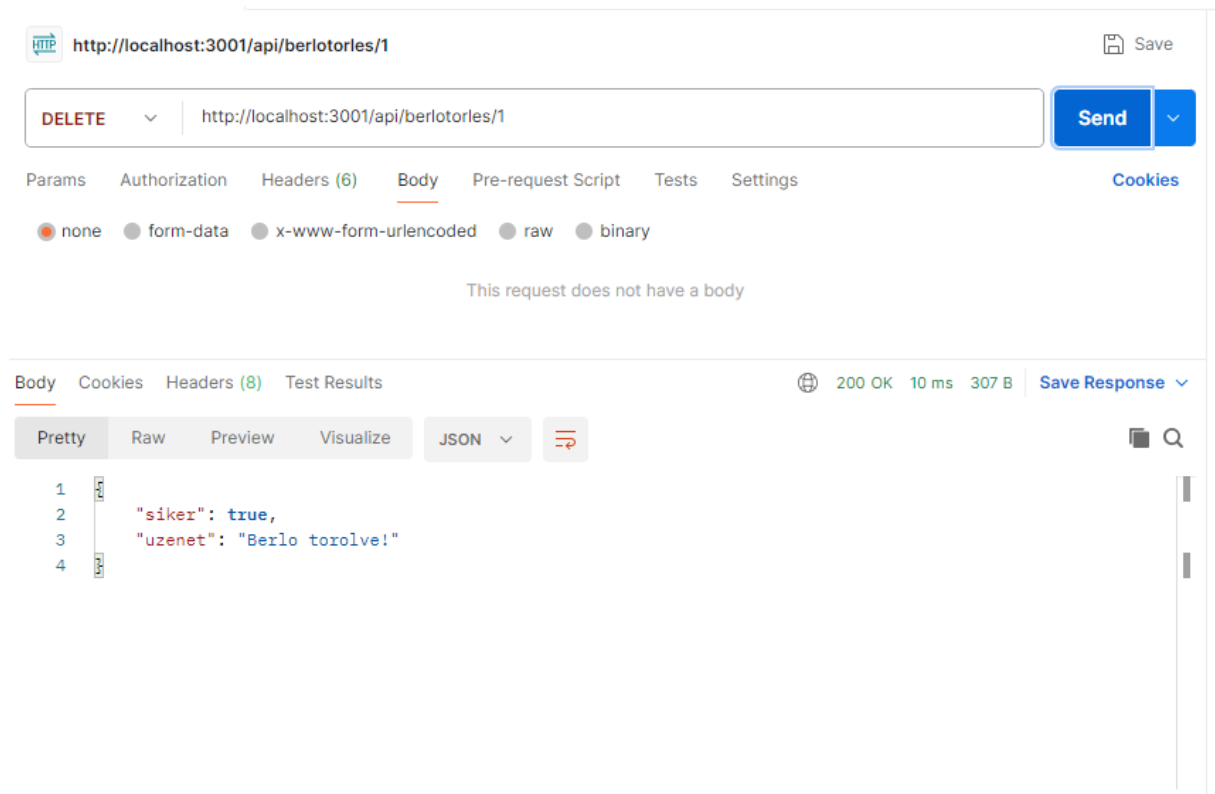
  }

});

module.exports = router;
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9



3. Részlet az berlotorles.js fájlból.

Bérlők listázás:

A bérlők listázása funkció fejlesztése során elkészítettem az összes bérlő adatainak lekérdezésére szolgáló API végpontot. A rendszer az adatbázisból lekéri a bérlők adatait és JSON formátumban továbbítja azokat a frontend számára. A fejlesztés célja az volt, hogy a felhasználók egyszerűen áttekinthető listában jeleníthessék meg a bérlőket. A lekérdezés során biztosítottam az adatbázis-kapcsolat megfelelő kezelését. Az elkészült végpont működését Postman segítségével sikeresen teszteltem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const adatok = await conn.query(
      "SELECT * FROM berlok"
    );

  }
```

```
        res.json(adatok);

    } catch (err) {

        res.status(500).json(err);

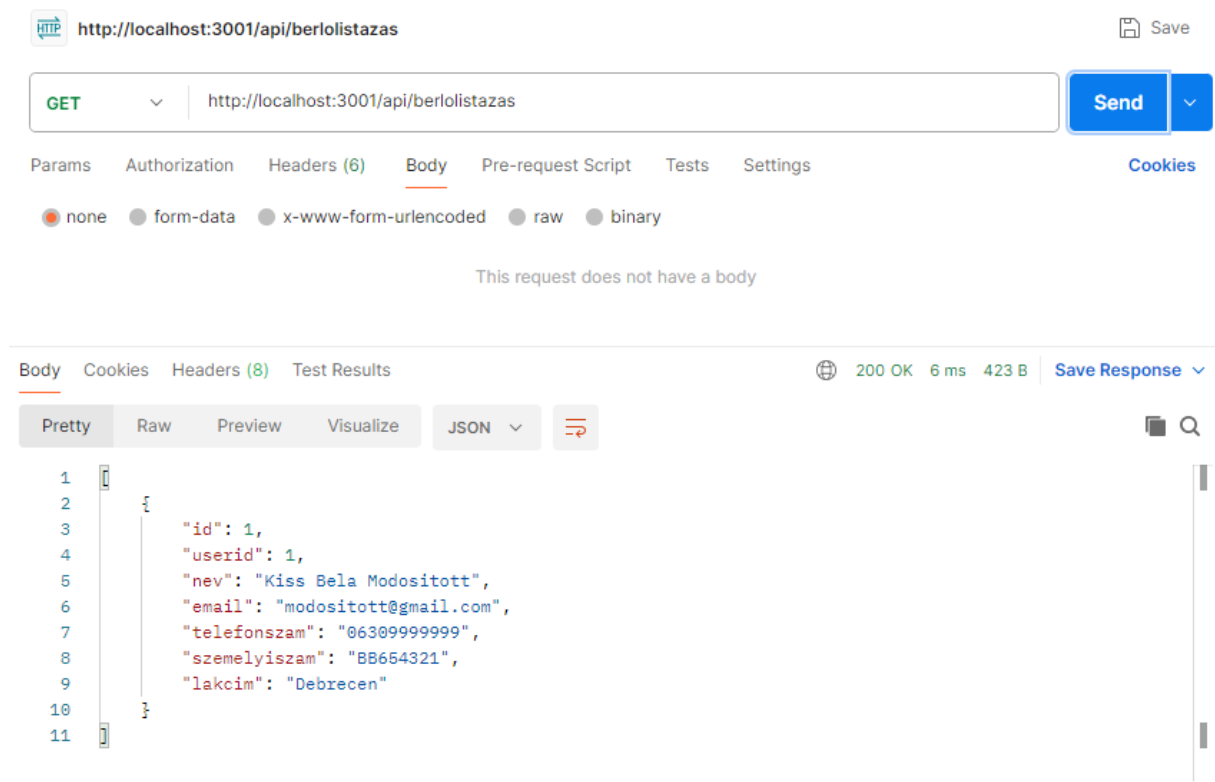
    } finally {

        if(conn) conn.release();

    }

});

module.exports = router;
```



The screenshot shows a REST client interface with the following details:

- URL: `http://localhost:3001/api/berlolistazas`
- Method: `GET`
- Body: `none`
- Response Status: `200 OK`
- Response Time: `6 ms`
- Response Size: `423 B`
- Response Body (JSON):

```
{  "id": 1,  "userid": 1,  "nev": "Kiss Bela Modositott",  "email": "modositott@gmail.com",  "telefonszam": "06309999999",  "szemelyiszam": "BB654321",  "lakcim": "Debrecen"}
```

4. Részlet az `berlolistazas.js` fájlból.

Bérlő hozzárendelése ingatlanhoz:

A bérlő hozzárendelése funkció fejlesztése során elkészítettem azt az API végpontot, amely lehetővé teszi egy bérlő és egy ingatlan összekapcsolását a rendszerben. A megvalósítás során a hozzárendelés egy új bérleti szerződés létrehozásával történt, amely tartalmazza az ingatlan és a bérlő azonosítóját, valamint a szerződéshez kapcsolódó adatokat. A rendszer eltárolja a szerződés kezdeti és végdátumát, a havi bérleti díjat és a szerződés aktuális állapotát is. A fejlesztés során biztosítottam, hogy a hozzárendelés csak létező bérlő és ingatlan rekordok között történhessen meg az adatbázis idegen kulcs kapcsolatain keresztül. Az elkészült funkció

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

segítségével nyilvántarthatóvá vált, hogy melyik bérlő melyik ingatlanhoz tartozik. A végpont működését Postman alkalmazás segítségével teszteltem, ahol ellenőriztem az adatok sikeres rögzítését és az adatbázisban történő megjelenését.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    ingatlanid,
    berloid,
    kezdetdatum,
    vegdatum,
    havidij,
    statusz
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `INSERT INTO szerzodesek
      (ingatlanid, berloid, kezdetdatum, vegdatum, havidij, statusz)
      VALUES (?, ?, ?, ?, ?, ?)`,
      [
        ingatlanid,
        berloid,
        kezdetdatum,
        vegdatum,
        havidij,
        statusz
      ]
    );

    res.json({
      siker: true,
      uzenet: "Bérlő sikeresen hozzárendelve az ingatlanhoz!"
    });

  } catch (err) {

    res.status(500).json(err);
  }
});
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
    } finally {  
  
        if(conn) conn.release();  
  
    }  
});  
  
module.exports = router;
```

HTTP <http://localhost:3001/api/berlohozzarendeles> Save

POST <http://localhost:3001/api/berlohozzarendeles> Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies

● none ● form-data ● x-www-form-urlencoded ● raw ● binary JSON Beautify

```
1 {  
2   "ingatlanid": 2,  
3   "berloid": 1,  
4 }
```

Body Cookies Headers (8) Test Results 200 OK 8 ms 341 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {  
2   "siker": true,  
3   "uzenet": "Bérlő sikeresen hozzárendelve az ingatlanhoz!"  
4 }
```

5. Részlet az `berlohozzarendeles.js` fájlból.